# CYBERSECURITY INTERNSHIP

**Name:** **Hammad Rasheed**

**ID:** **DHC-5590**

# Week 2

## Implementing Security Measures

### 1. Input Validation & Sanitization

```
const validator = require('validator');

app.post('/api/Users', async (req, res) => {
  const { email, password } = req.body;

  if (!email || !validator.isEmail(email)) {
    return res.status(400).send('Invalid email format');
  }

  const sanitizedEmail = validator.escape(email);
  const sanitizedPassword = validator.escape(password);
```

```
  // Continue safely...
});
```

## 2. Password Hashing with bcrypt

### Install

```
npm install bcrypt
```

### Secure Registration

```
const bcrypt = require('bcrypt');
const saltRounds = 10;

const hashedPassword = await bcrypt.hash(password,
saltRounds);
```

### Secure Login

```
const match = await bcrypt.compare(password, user.password);
```

## 3. JWT Token-Based Authentication

### Install

```
npm install jsonwebtoken
```

### Generate JWT

```
const jwt = require('jsonwebtoken');

const token = jwt.sign(
```

```
  { id: user.id, email: user.email, role: user.role },
  JWT_SECRET,
  { expiresIn: '1h' }
);
```

### Verify JWT

```
jwt.verify(token, JWT_SECRET, (err, decoded) => {
  if (err) return res.status(403).send('Invalid token');
});
```

## 4. Secure HTTP Headers with Helmet

### Install

```
npm install helmet
```

### Usage

```
const helmet = require('helmet');

app.use(helmet());
```

### Important Headers

- X-Frame-Options
- CSP
- HSTS
- X-Content-Type-Options

## 5. SQL Injection Prevention

### Vulnerable Code

```
const query = `SELECT * FROM Users
WHERE email='${email}' AND password='${password}'`;
```

### Secure Version

```
const query = 'SELECT * FROM Users WHERE email = ? AND
password = ?';

db.query(query, [email, hashedPassword]);
```

### Better Approach

```
const user = await User.findOne({
  where: { email }
});
```

---